JAVA Programming (CST015)

Lecture 5: Conditional Statements in JAVA

Department of Computer Science and Engineering

National Institute of Technology, Srinagar, Jammu and Kashmir

March 09, 2026

The if Statement

- The Java ***if statement*** has the following syntax:

```
if (boolean-condition)
    statement;
```

- If the Boolean condition is true, the statement is executed; if it is false, the statement is skipped
- This provides basic decision making capabilities

The if-else Statement

- An *else clause* can be added to an `if` statement to make an *if-else statement*

```
if ( condition )
     statement1;
else
     statement2;
```

- If the *condition* is true, *statement1* is executed; if the condition is false, *statement2* is executed
- One or the other will be executed, but not both

Output?

```
int a =10;
if(a<9)
    System.out.println("Value is less than 9");
    ++a;
    System.out.println(“Value of a is : "+a);
```

Indentation Revisited

- Remember that indentation is for the human reading and is ignored by the Java compiler

```
int a =10;
if(a<9)
   System.out.println("Value is less than 9");
   ++a;
   System.out.println("I should not be executed"+a);
```

Despite what is implied by the indentation, the increment will occur. No matter, the if condition is true or not.

Block Statements

- Several statements can be grouped into a *block statement* delimited by braces

```
if (total > MAX)
{
   System.out.println ("Error!!");
   errorCount++;
}
```

Now the increment will only occur when the if condition is true

- A block statement can be used wherever a statement is called for in the Java syntax

The else if Statement

- Use the **else if** statement to specify a new condition if the first condition is false.

```
if (condition1) {
        // block of code to be executed if condition1 is true
        }
 else if (condition2) {
        // block of code to be executed
                if the condition1 is false and condition2 is true
        }
 else {
        // block of code to be executed
        if the condition1 is false and condition2 is false
        }
```

Example

```
public class Vote
{
        public static void main(String[] args)
                {
                        int a=19;
                        if (a==18)
                        {
                                System.out.println("age is 18");
                        }
                        else if (a>18)
                        {
                                System.out.println("age is greater than 18");
                        }
                        else
                        {
                                System.out.println("age is less than 18");
                        }
        }
}
```

If if-else without default else case

```
public class Vote
{
        public static void main(String[] args)
                {
                        int a=19;
                        if (a==18)
                        {
                                System.out.println("age is 18");
                        }
                        else if (a>18)
                        {
                                System.out.println("age is greater than 18");
                        }
        }
}
```

Output?

```
public class Vote
{
public static void main(String[] args)
{
        int a = 10, b = 5;

        if(a > b)
                System.out.print("A is greater");
        else;
                System.out.print("B is greater");

}
}
```

Solution to Previous Question

```
public class Vote
{
public static void main(String[] args)
{
        int a = 10, b = 5;

        if(a > b)
                System.out.println("A is greater");
        else;
                System.out.print("B is greater");
}
}
```

Output will be:

A is greater

B is greater

because the semicolon after the else statement creates an empty block,
so the next line ("System.out.print("B is greater")") is executed unconditionally, regardless of the condition in the if statement.

Output?

```
public class Vote
{
public static void main(String[] args)
{

        int a = 10, b = 5, c = 2;
        if(a > b || ++c == 3)
                System.out.print("A is greater");
        else
                System.out.print("B is greater");
                System.out.print(c);
}
}
```

Solution to Previous Question

```
public class Vote
{
public static void main(String[] args)
{

        int a = 10, b = 5, c = 2;
        if(a > b || ++c == 3)
                System.out.print("A is greater");
        else
                System.out.print("B is greater");
                System.out.print(c);
}
}
```

Output:

A is greater

2

This is because the || operator (logical OR) is used here, which evaluates the left-hand side first. Since a > b is true, the right-hand side (++c == 3) is not evaluated, and the if block is executed. The value of c remains 2 because ++c == 3 is not executed.

Output?

```
public class Vote
{
public static void main(String[] args)
{

        int a = 10, b = 5, c = 2;
        if (a > b && ++c == 3)
                System.out.print("a is greater");
        else
                System.out.print("b is greater");
                System.out.print(c);

}
}
```

Solution to Previous Question

```
public class Vote
{
public static void main(String[] args)
{

        int a = 10, b = 5, c = 2;
        if (a > b && ++c == 3)
                System.out.print("a is greater");
        else
                System.out.print("b is greater");
                System.out.print(c);

}
}
```

Output: a is greater 3

because the && operator (logical AND) is used here, and both conditions must be true for the if block to be executed. Since a > b is true, the right-hand side (++c == 3) is also evaluated, and the value of c is incremented to 3.

Program:
Checking whether a number is Even or not using Methods

Switch Statements in Java

- Use the switch statement to select one of many code blocks to be executed.

```
switch(expression) {
        case x:
                // code block
                break;
        case y:
                // code block
                break;
        default:
                // code block
}
```

Switch Statements in Java

This is how it works:

- The switch expression is evaluated once.
- The value of the expression is compared with the values of each case.
- If there is a match, the associated block of code is executed.
- The break and default keywords are optional,

```java
int day = 4;
switch (day) {
    case 1:
        System.out.println("Monday");
        break;
    case 2:
        System.out.println("Tuesday");
        break;
    case 3:
        System.out.println("Wednesday");
        break;
    case 4:
        System.out.println("Thursday");
        break;
    case 5:
        System.out.println("Friday");
        break;
    case 6:
        System.out.println("Saturday");
        break;
    case 7:
        System.out.println("Sunday");
        break;
}
```

The break Keyword

- When Java reaches a break keyword, it breaks out of the switch block.
- This will stop the execution of more code and case testing inside the block.
- When a match is found, and the job is done, it's time for a break. There is no need for more testing.

The default Keyword

- The default keyword specifies some code to run if there is no case match

Output?

```
public class Vote
{
public static void main(String[] args)
{

char ch = 'A';
switch (ch) {
   case 'a':
      System.out.print("a");
   case 'A':
      System.out.print("A");
   case 'b':
      System.out.print("b");
   default:
      System.out.print("Default");
}
}
}
```

Output?

```
public class Vote
{
public static void main(String[] args)
{
int a = 10, b = 5;
switch (a - b) {
   case 2:
      System.out.print("Two");
      break;
   case 5:
      System.out.print("Five");
      break;
   case 10:
      System.out.print("Ten");
      break;
   default:
      System.out.print("Default");
}
}}
```